

VSD HW1

Instructor: Lih-Yih Chiou

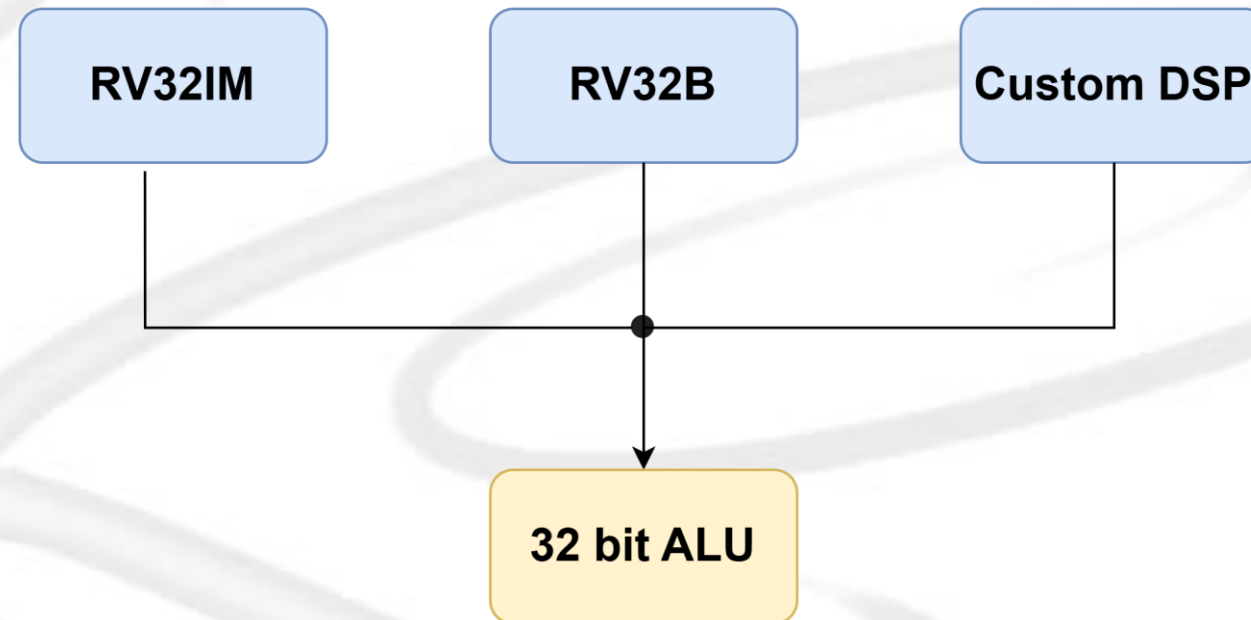
TA: Grace

Date: 9/14/2026



Introduction to ALU (RV32IMB + DSP)

- This homework extends the conventional RV32IM ALU with:
 - RV32B Standard Extension
 - Custom DSP Extension



Hardware Description (1/4) — I/O Port Description

Signal	Type	Width	Description
instruction_i	In	32 bits	32-bit RISC-V instruction used to decode the ALU operation
src_A_i	In	32 bits	First source operand (corresponds to the data of rs1)
src_B_i	In	32 bits	Second source operand (corresponds to the data of rs2)
result_o	Out	32 bits	ALU output result

- $\text{src_A_i} = \$(\text{rs1})$, $\text{src_B_i} = \$(\text{rs2})$, $\text{result_o} = \$(\text{rd})$
- The ALU operation is determined by decoding the corresponding fields in **instruction_i**.
- For R-type & M Standard instruction format:

31	25	24	20	19	15	14	12	11	7	6	0
funct7		rs2		rs1		funct3		rd		opcode	

Hardware Description (2/4) — RV32IM Instructions

	Inst.	funct7	funct3	Description
0	ADD	0000000	000	$rd = rs1 + rs2$
1	SUB	0100000	000	$rd = rs1 - rs2$
2	SLL	0000000	001	$rd = rs1 \ll (rs2[4:0])$
3	SLT	0000000	010	$rd = (rs1 < rs2) ? 1 : 0$ (signed)
4	SLTU	0000000	011	$rd = (rs1 < rs2) ? 1 : 0$ (unsigned)
5	XOR	0000000	100	$rd = rs1 \wedge rs2$
6	SRL	0000000	101	$rd = rs1 \gg (rs2[4:0])$ (logical)
7	SRA	0100000	101	$rd = rs1 \ggg (rs2[4:0])$ (arithmetic)
8	OR	0000000	110	$rd = rs1 \mid rs2$
9	AND	0000000	111	$rd = rs1 \& rs2$
10	MUL	0000001	000	$rd = (rs1 * rs2)[31:0]$
11	MULH	0000001	001	$rd = (\text{signed}(rs1) * \text{signed}(rs2))[63:32]$
12	MULHSU	0000001	010	$rd = (\text{signed}(rs1) * \text{unsigned}(rs2))[63:32]$
13	MULHU	0000001	011	$rd = (\text{unsigned}(rs1) * \text{unsigned}(rs2))[63:32]$

opcode = 0110011

Hardware Description (3/4) — RV32B Standard Extension

	Inst.	funct7	[24:20]	[19:15]	funct3	[11:7]	opcode	Description
0	CLZ	0110000	00000	rs1	001	rd	0010011	rd = count_leading_zeros(rs1)
1	CTZ	0110000	00001	rs1	001	rd	0010011	rd = count_trailing_zeros(rs1)
2	CPOP	0110000	00010	rs1	001	rd	0010011	rd = count_ones(rs1)
3	ROL	0110000	rs2	rs1	001	rd	0110011	rd = rotate_left(rs1, rs2[4:0])
4	ROR	0110000	rs2	rs1	101	rd	0110011	rd = rotate_right(rs1, rs2[4:0])
5	MIN	0000101	rs2	rs1	100	rd	0110011	rd = (signed(rs1) < signed(rs2)) ? rs1 : rs2
6	MAX	0000101	rs2	rs1	110	rd	0110011	rd = (signed(rs1) > signed(rs2)) ? rs1 : rs2
7	MINU	0000101	rs2	rs1	101	rd	0110011	rd = (unsigned(rs1) < unsigned(rs2)) ? rs1 : rs2
8	MAXU	0000101	rs2	rs1	111	rd	0110011	rd = (unsigned(rs1) > unsigned(rs2)) ? rs1 : rs2
9	BSET	0010100	rs2	rs1	001	rd	0110011	rd = rs1 (1 << rs2[4:0])
10	BCLR	0100100	rs2	rs1	001	rd	0110011	rd = rs1 & ~(1 << rs2[4:0])
11	BEXT	0100100	rs2	rs1	101	rd	0110011	rd = (rs1 >> rs2[4:0]) & 1

Hardware Description (4/4) — DSP Custom Extension

Inst.	funct7	funct3	opcode	Description
DOTP4	0000000	000	0001011	$rd = \sum \text{signed}(rs1[8i+7:8i]) \times \text{signed}(rs2[8i+7:8i]), i = 0 \sim 3$
SADD8	0000000	001	0001011	$rd[8i+7:8i] = \text{SAT8}(\text{signed}(rs1[8i+7:8i]) + \text{signed}(rs2[8i+7:8i])), i = 0 \sim 3$
SSUB8	0000000	010	0001011	$rd[8i+7:8i] = \text{SAT8}(\text{signed}(rs1[8i+7:8i]) - \text{signed}(rs2[8i+7:8i])), i = 0 \sim 3$
ABS	0000000	011	0001011	$rd = (rs1 == 32'h80000000) ? 32'h7FFFFFFF : ((\text{signed}(rs1) < 0) ? -\text{signed}(rs1) : rs1)$
CLIP8	0000000	100	0001011	$rd = (\text{signed}(rs1) > 127) ? 127 : (\text{signed}(rs1) < -128) ? -128 : \text{signed}(rs1)$

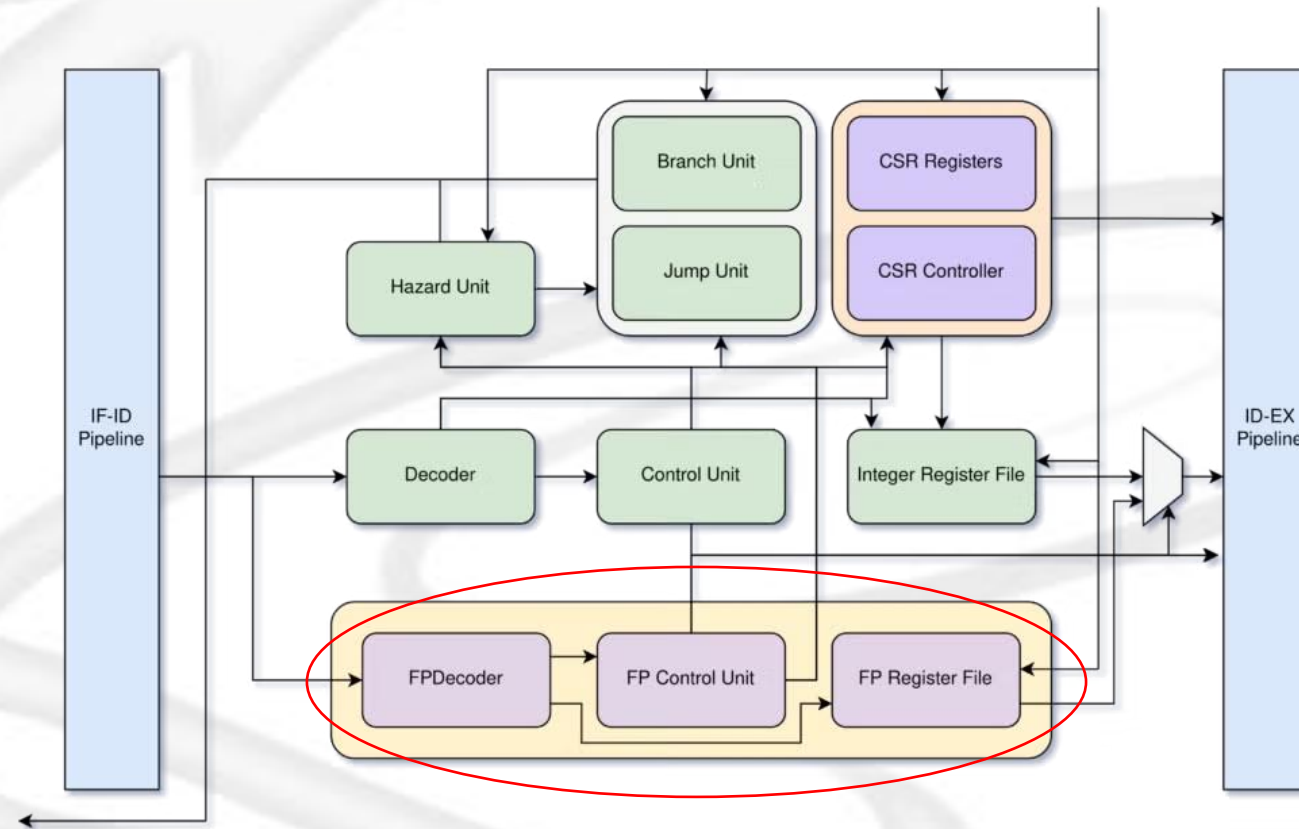
Hardware Description (4/4) — DSP Custom Extension

<Note>

- ❑ DSP instructions use an **R-type-like custom encoding format**. For unary operations such as ABS and CLIP8, the rs2 field is fixed to 00000.
- ❑ For the minimum 32-bit signed value (32'h80000000), **ABS saturates the result to 32'h7FFFFFFF**.
- ❑ DOTP4, SADD8, and SSUB8 treat each 8-bit element as a **signed two's-complement value**. **Eg. 8'hFF is interpreted as -1, not 255**
- ❑ Saturation for SADD8 and SSUB8 is performed independently for each 8-bit lane. **Eg. 100 + 50 → 127, while 10 + 20 → 30 in another lane.**
- ❑ CLIP8 returns a 32-bit **sign-extended** result.

Introduction to FPU

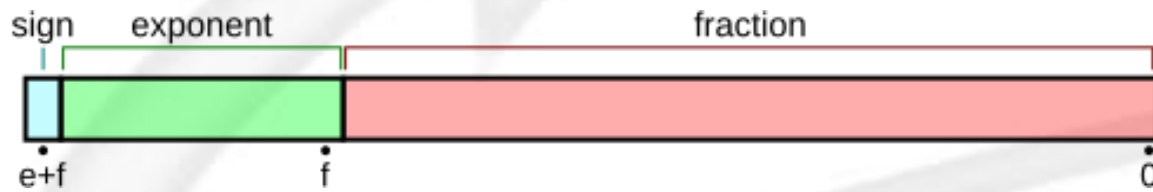
- In order to design an IMF-CPU
 - ➔ We need an FPU.



Floating Point definition

Single precision and double precision

- Single precision(32bits) : 1(sign) + 8 (exponent) + 23 (frac)
- Double precision(64 bits) : 1(sign) + 11(exponent) + 52 (frac)



$$\text{Value} = (-1)^S * (1.F) * 2^{E-127}$$

Data	0x40400000
S	0
E	(10000000)128
F	1.01(1.25)
Value	2.5

Mantissa & GRS

- Mantissa(fraction part)
 - ➔ The normalized form is **1.F**
- GRS(Guard 、 Round 、 Sticky)
 - ➔ To align the exponents, we need to shift the mantissa.
 - ➔ G (Guard bit): the **first bit** shifted out of the mantissa.
 - ➔ R (Round bit): the **second bit** shifted out of the mantissa.
 - ➔ S (Sticky bit): if any bit after R is 1, then S is set to 1.

Fraction(1.F)

24bits

G|R|S

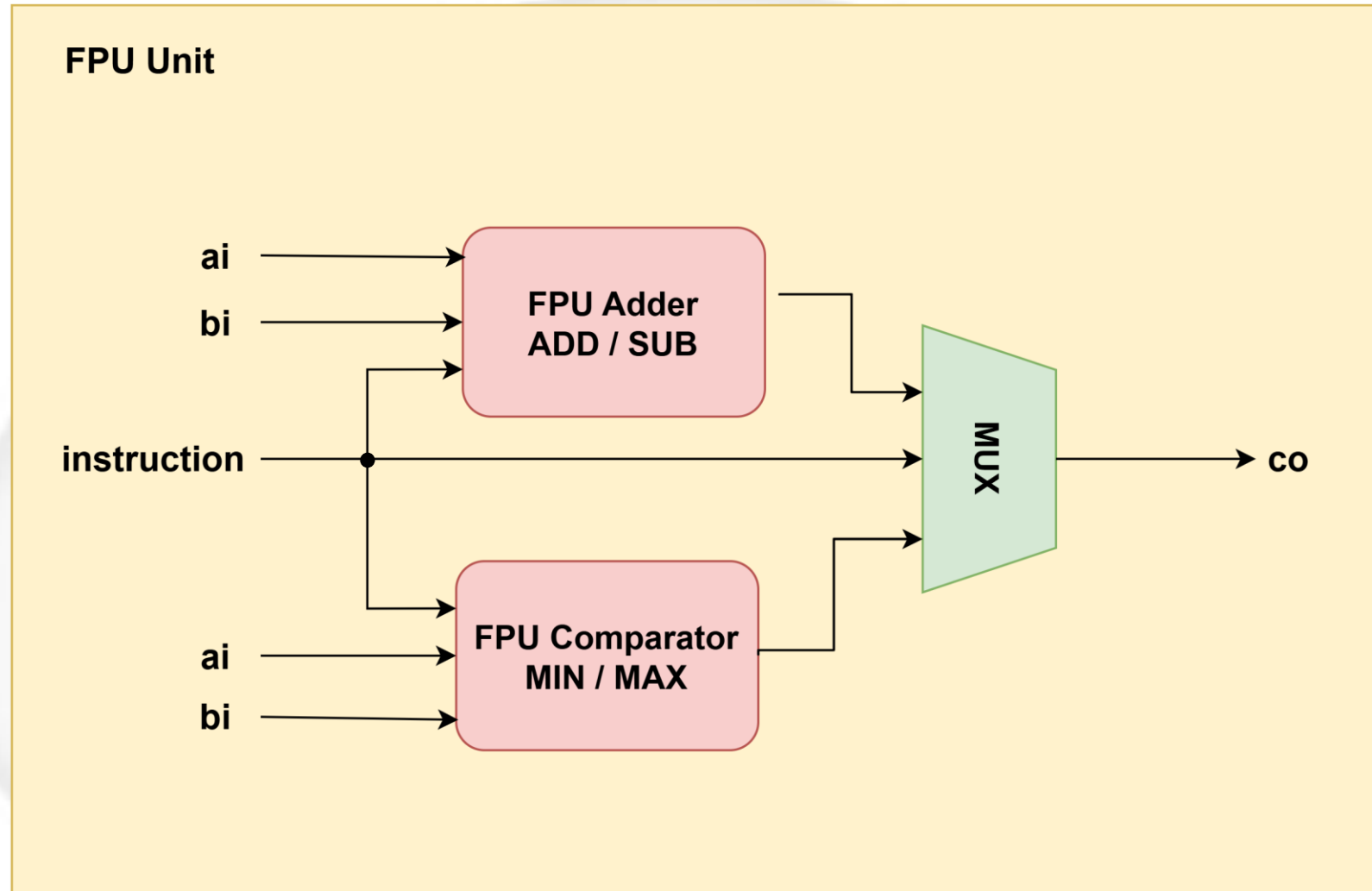
3bits

Rounding Policy(round to even)

□ Example

Round-to-even			
GRS	LSB	Action	Example
0xx	X	Discard	0.100_010 $\rightarrow$ 0.100
110 、 101 、 111	X	Round up	0.100_110 $\rightarrow$ 0.111 0.100_101 $\rightarrow$ 0.111
100	1	Round up	0.101_100 $\rightarrow$ 0.110
100	0	Discard	0.100_100 $\rightarrow$ 0.100

Hardware Description (1/4) — FPU Architecture



Hardware Description (2/4) — I/O Port Description

Signal	Type	Bits	Description
clk	Input	1	Clock signal
rst	Input	1	Reset signal
enable	Input	1	Enable signal
ai	Input	32	First number
bi	Input	32	Second number
instruction	Input	2	0 : min 1 : max 2 : add 3 : sub
co	Output	32	Output answer
valid	Output	1	Output valid

Add and Sub Steps (1/3) - Unpacking

- Unpack data to SEF and find special cases (NaN, Inf, Zero).

Type	Exponent	Fraction
NaN	255	Non-zero
Inf	255	Zero
Zero	0	zero

Add and Sub Steps(2/3) – ADD/SUB

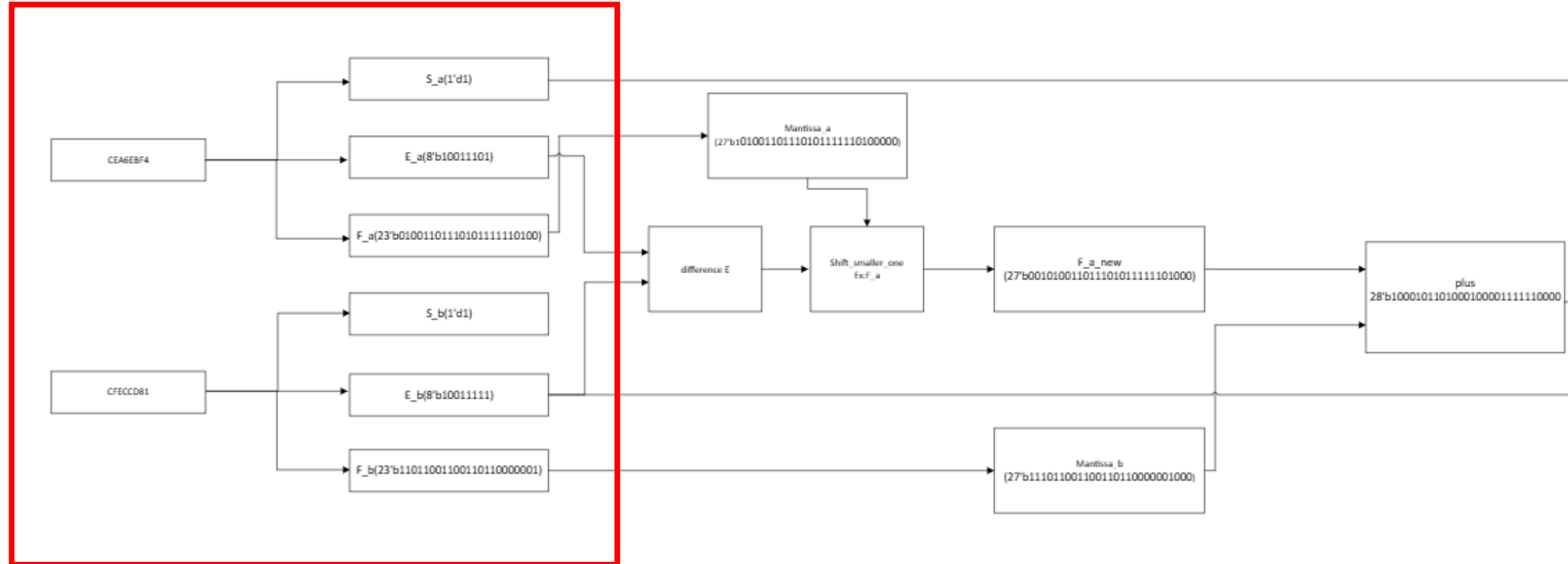
- ❑ Shift the smaller mantissa right, **shift make exponents equal**
 - ➔ $\Delta E = |E_a - E_b|$
 - ➔ Right-shift the mantissa of the operand with the smaller exponent by ΔE
- ❑ Perform mantissa addition or subtraction depending on operand signs and the instruction. Below is special case

Operation	Result
Any nan in operand	Nan (32'hFFC00000)
Any inf in operand (except for sign mismatch)	Inf(32'hFF800000 or 32'h7F800000)
Sign mismatch inf (+inf + -inf)	Nan (32'hFFC00000)
0 + 0 (sign depend on & each other)	+0(32'h00000000) or -0(32'h80000000)
Only a 0 in operand	Return another one

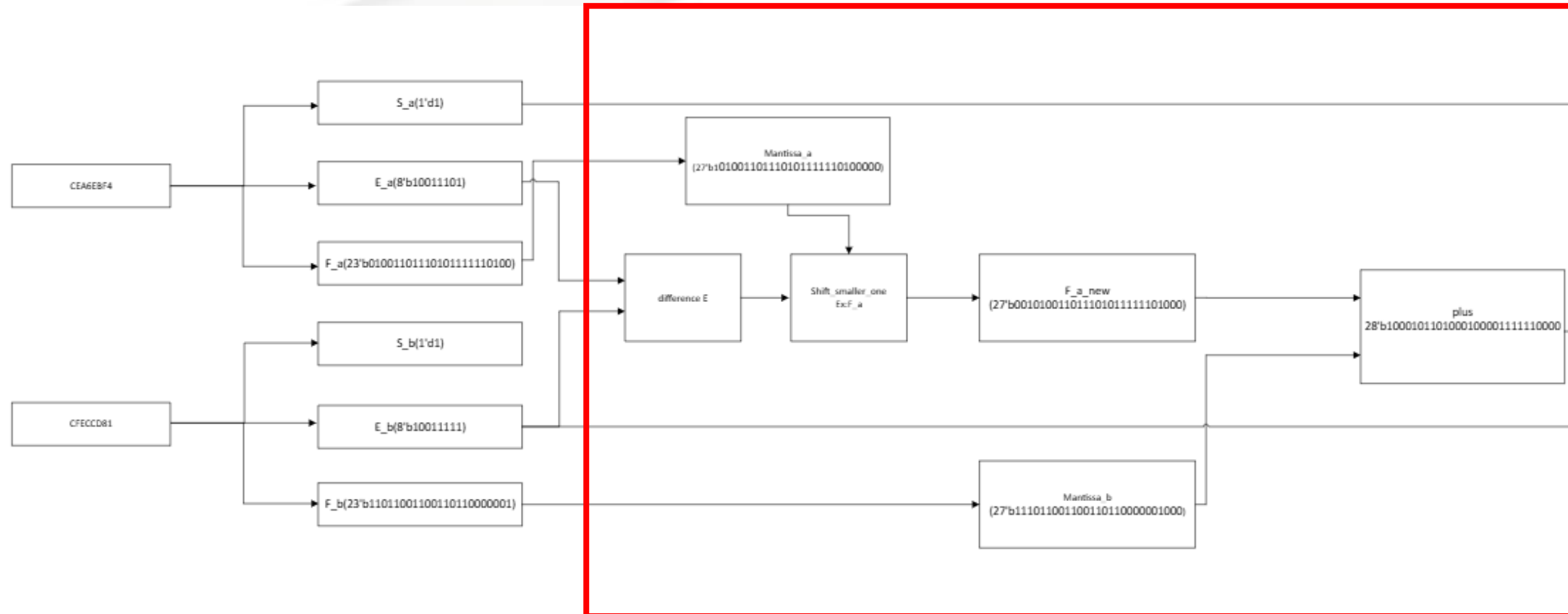
Add and Sub Steps(3/3) – Normalize & Round

- ❑ Handle carry (shift right, exponent +1) or cancellation (shift left, exponent $-k$).
- ❑ Apply rounding policy using GRS bits.
- ❑ Repack into 32bits data and output the result.

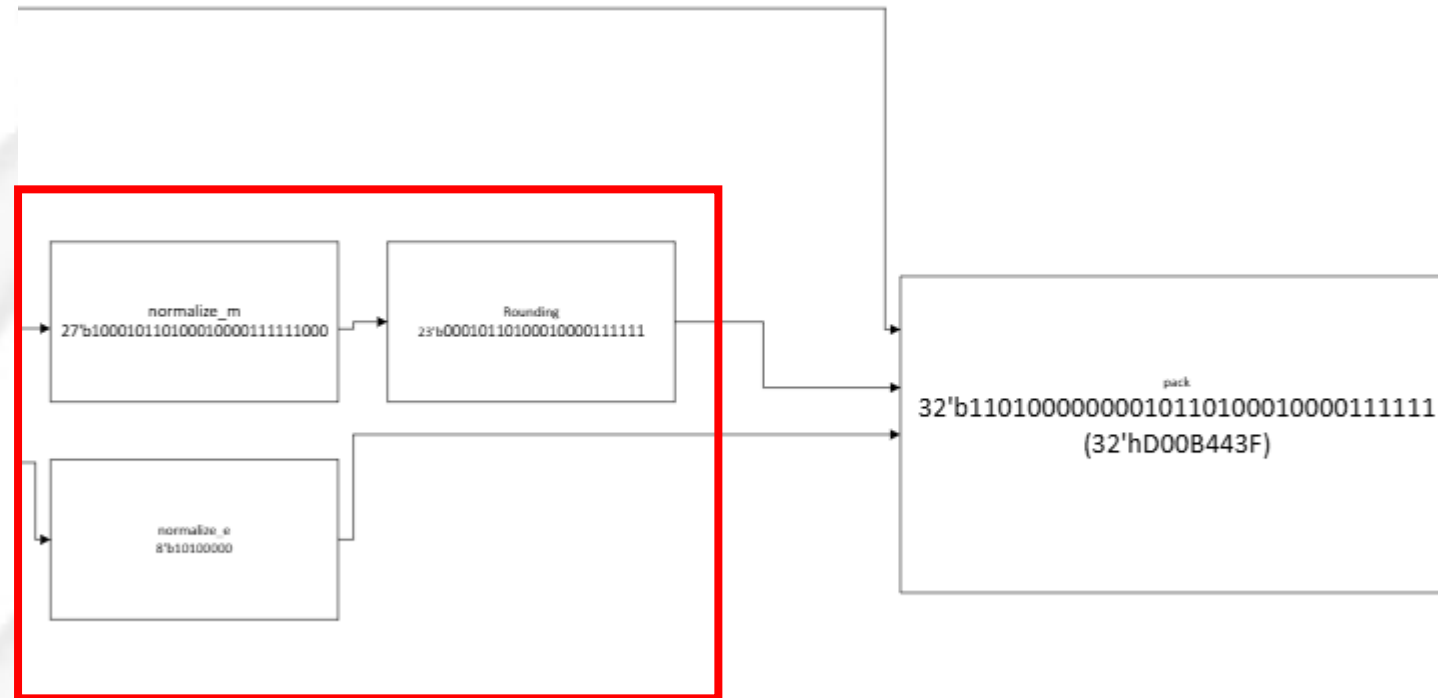
Example(1/3) - Unpacking



Example(2/3)– Add/Sub

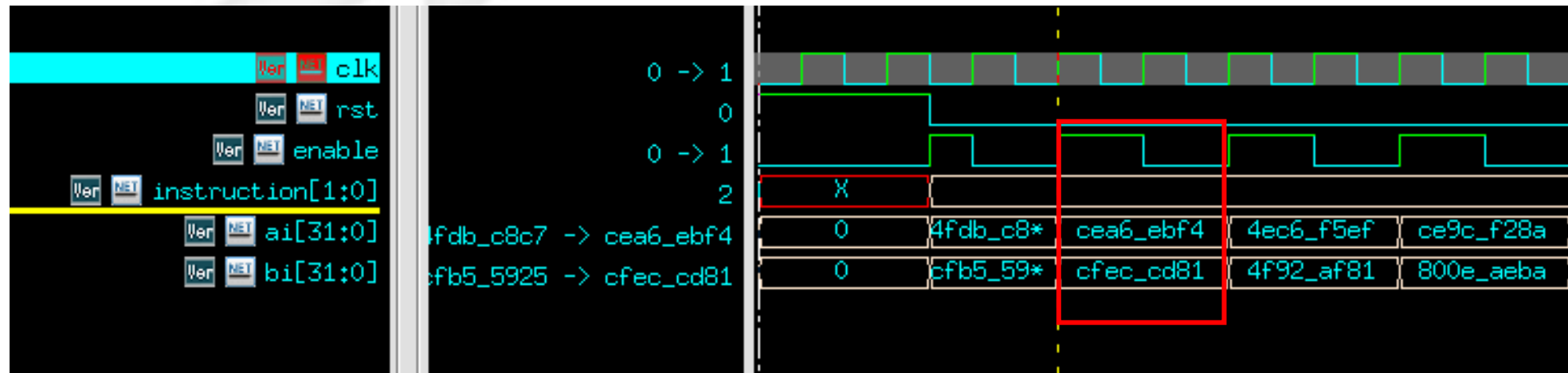


Example(3/3) – Normalize & Rounding



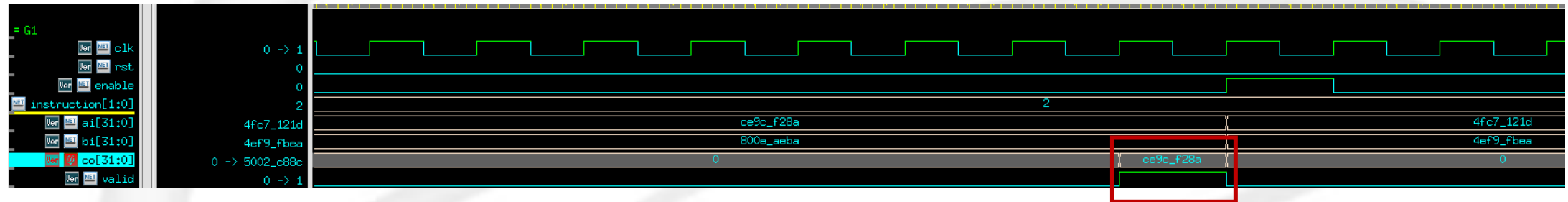
Data transfer(1/2)

- After reset low, pull high enable and feed data in!!



Data transfer(2/2)

- Wait valid signal pull high check the answer with golden, and next cycle new data feed into FPU.



Homework

□ Prob. A—ALU

Design and implement a **32-bit ALU** supporting selected **RV32IMB instructions** and **custom DSP extensions**.

- RV32IM: 14 instructions
- RV32B: 12 instructions
- DSP: 5 custom instructions
- **Total: 31 instructions**

□ Prob. B—FPU

Design and implement a complete **32-bit FPU** based on the **IEEE-754 single-precision format**.

- ADD / SUB
- MIN / MAX
- Special-case handling
- Normalization and GRS rounding

Both designs must pass the provided simulation patterns and hidden test cases.

File hierarchy

□ Homework submission must follow the file hierarchy!

- ➔ HW1_StudentID.tar
 - ◆ HW1_StudentID
 - StudentID.pdf
 - Src
 - ALU.sv
 - FPU.sv
 - Other user-defined .sv files
 - Sim
 - FPU_tb.sv
 - ALU_tb.sv
 - Superlint.tcl
 - Pattern
 - pattern_alu.csv
 - pattern_fpu.csv

Report must be in .pdf format

Commands and Friendly reminder

□ VCS Commands

➔ Prob.A

◆ `vcs -R -full64 -sverilog ALU_tb.sv -debug_access+all +define+FSDB`

➔ Prob.B

◆ `Vcs -R -full64 -sverilog FPU_tb.sv -debug_access+all +define+FSDB`

➔ Superlint

◆ `jg -superlint superlint.tcl`

Grading policy(1/2)

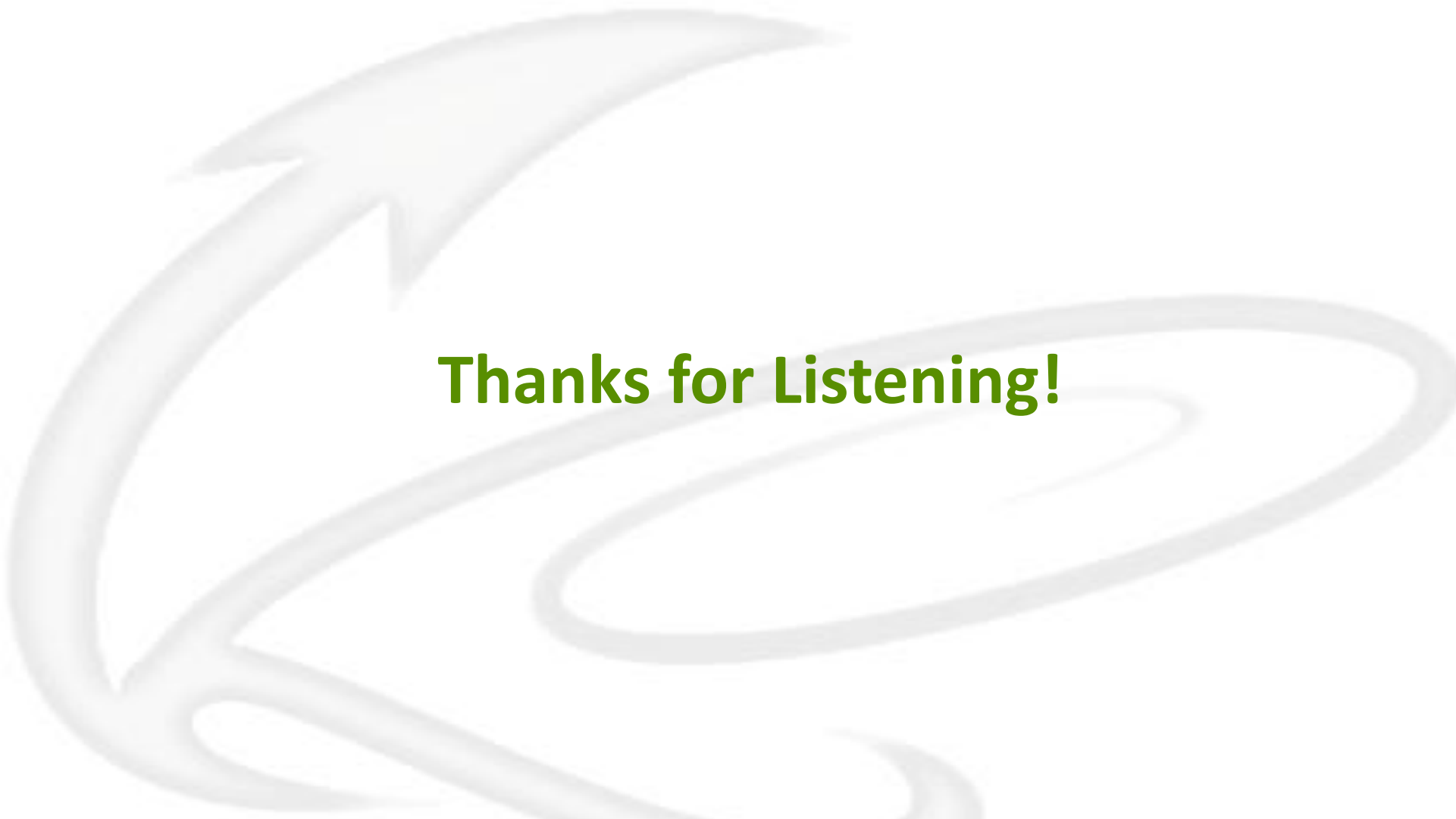
- Prob. A(40%) : ALU
- Prob. B(40%) : FPU
- Report (20%)
 - ➔ The more complete you elaborate your design, the higher your grade is.
 - ➔ Do not copy and paste your whole code in the report, however, screenshot of some portion that help writing the report is permitted.
- Superlint coverage $\leq 80\%$ (-10%)

Grading policy(2/2)

- ❑ Submission that **fails to follow the file hierarchy and format** will result in a **10-credit** reduction.
- ❑ Duration: **10/5/2026**.
- ❑ Compress your homework in **.tar format** and upload it to Moodle.
 - ➔ Follow the file hierarchy for homework submission.
 - ➔ Ensure your code can be compiled and simulated in the SoC Lab environment.
 - ➔ Use **%tar -cvf StudentID_HW1.tar StudentID_HW1** to compress your folder. The compressed file should be named as **"StudentID_HW1.tar"** (e.g. **E24xxxxxx_HW1.tar**)

Late Submission Policy

- ❑ Late submission within 24hr $\rightarrow$ score $\times$ 0.8
- ❑ Late submission within 48hr $\rightarrow$ score $\times$ 0.6
- ❑ Late submission within 72hr $\rightarrow$ score $\times$ 0.4
- ❑ Late submission more than 72hr $\rightarrow$ assignment will NOT be accepted.
(score $\times$ 0)



Thanks for Listening!